Homework Solutions: Type Inference & Program Verification

Mohammad Abu Shariha
PSU ID: mma7020@psu.edu
December 8, 2025

Problem 1: Type Inference I

1. For $f1 = \lambda x.\ x - 1$, the subtraction operator requires arguments to be integers.

$$f1 : \text{int} \to \text{int}$$

2. For $f2 = \lambda x.\lambda y.\ x > y$, the comparison operator $>$ takes two integers and returns a boolean.

$$f2 : \text{int} \to (\text{int} \to \text{bool})$$

3. For $f3 = \lambda x.\lambda y.\ (x\ 1\ y)$: The term $(x\ 1\ y)$ implies x is a function applied to the integer constant 1, returning another function that is applied to y. Let y have type τ and the result of the expression have type σ. The constant 1 forces the first argument of x to be int. Thus:

$$f3 : (\text{int} \to (\tau \to \sigma)) \to (\tau \to \sigma)$$

Problem 2: Type Inference II

Term: $(\lambda x.\lambda y.\ y \wedge \neg y)$ true

Answer: The type of the term is **bool** $\to$ **bool**.

Types of Variables:

- x: **bool** (Constrained by the application to `true`).
- y: **bool** (Constrained by logical operators $\wedge, \neg$).

Type Derivation Tree: We verify the type using the inference rules (T-VAR, T-NOT, T-AND, T-ABS, T-APP). Let context $\Gamma = \{x : \text{bool}, y : \text{bool}\}$.

$$\dfrac{\dfrac{\dfrac{}{\Gamma \vdash y : \text{bool}}\,(\text{T-VAR}) \quad \dfrac{\dfrac{}{\Gamma \vdash y : \text{bool}}\,(\text{T-VAR})}{\Gamma \vdash \neg y : \text{bool}}\,(\text{T-NOT})}{\Gamma \vdash y \wedge \neg y : \text{bool}}\,(\text{T-AND})}{\Gamma \setminus \{y\} \vdash \lambda y.\ y \wedge \neg y : \text{bool} \to \text{bool}}\,(\text{T-ABS})$$

The full term is an application. Let $M = (\lambda x.\lambda y.y \wedge \neg y)$. The derivation for the full term applies M to `true`:

$$\dfrac{\dfrac{[\text{Derivation above}]}{\emptyset \vdash \lambda x.\lambda y.y \wedge \neg y : \text{bool} \to (\text{bool} \to \text{bool})}\,(\text{T-ABS}) \quad \dfrac{}{\emptyset \vdash \text{true} : \text{bool}}}{\emptyset \vdash (\lambda x.\lambda y.\ y \wedge \neg y)\ \text{true} : \text{bool} \to \text{bool}}\,(\text{T-APP})$$

Problem 3: Program Verification I

Program:

```
u := u + 3;
v := 2*v - u;
w := w + v;
```

We want final postcondition $Q : w = 0$.

1. Construct a Precondition

We work backward from the postcondition:

1. After $w := w + v$:

$$w + v = 0 \implies w = -v$$

2. Before $v := 2v - u$: Substituting $v \to (2v - u)$ into $w = -v$:

$$w = -(2v - u) = u - 2v$$

3. Before $u := u + 3$: Substituting $u \to (u + 3)$ into $w = u - 2v$:

$$w = (u + 3) - 2v$$

Thus the precondition is:

$$P : w = u + 3 - 2v$$

2. Weakest Precondition

Using the derivations above:

$$wp(\text{program},\ w = 0) \equiv (w = u + 3 - 2v)$$

3. Program Reasoning

(a) Restrictions on Initial Values

The restriction imposed on the initial values is that they must satisfy the equation derived in the weakest precondition. **Restriction:** The initial state must satisfy $w = u + 3 - 2v$ for the program to terminate with $w = 0$.

(b) New Postcondition: $w \geq 0$

We perform backward evaluation with the inequality:

1. After last stmt: $w + v \geq 0$ 2. After 2nd stmt: $w + (2v - u) \geq 0$ 3. After 1st stmt:

$$w + (2v - (u + 3)) \geq 0$$

Thus the new weakest precondition is:

$$w \geq u + 3 - 2v$$

Problem 4: Program Verification II

Program:

```
i := 0;
sum := 0;
while (i < n) do
    sum := sum + (2*i + 1);
    i := i + 1;
end
```

Given:

$$P : n \geq 0, \qquad Q : sum = n^2.$$

1. Loop Invariant

We use the mathematical identity that the sum of the first i odd numbers is i^2:

$$1 + 3 + 5 + \cdots + (2i - 1) = i^2$$

Thus the invariant is:

$$I : sum = i^2 \ \wedge \ 0 \leq i \leq n$$

2. Verification Conditions

(a) Initialization

We must show that the invariant holds before the loop starts. Initially $i = 0$ and $sum = 0$.

- $sum = 0 = 0^2$ (Holds)
- $0 \leq 0 \leq n$ (Holds since precondition $n \geq 0$)

The invariant holds initially.

(b) Maintenance

We must show that $\{I \wedge (i < n)\}$ Body $\{I\}$. We calculate the Weakest Precondition (WP) of the loop body with respect to I:

$$\text{Body: } sum := sum + (2i + 1); \ i := i + 1;$$

$$WP(\text{Body}, I) = I[i \to i + 1][sum \to sum + 2i + 1]$$

Substituting into I:

$$WP = (sum + 2i + 1 = (i + 1)^2) \ \wedge \ (0 \leq i + 1 \leq n)$$

We assume I ($sum = i^2$) and the loop guard ($i < n$).

1. **Sum:** $sum + 2i + 1 = i^2 + 2i + 1 = (i + 1)^2$. This matches the WP.
2. **Bounds:** Since $0 \leq i < n$, we know $i + 1 \leq n$ and $0 \leq i + 1$. This matches the WP.

Thus, the invariant is preserved.

(c) Termination

The loop terminates when the guard is false: $\neg(i < n) \implies i \geq n$. Combined with the invariant $0 \leq i \leq n$, this implies $i = n$. Substituting $i = n$ into the invariant $sum = i^2$:

$$sum = n^2$$

This matches the postcondition Q.